**React State Management and Event Handling | Lesson 3**

# Practice: React Weather App

## Instructions

You will create a React application where:

1. The main page lists a few city options (e.g., "New York," "London," "Tokyo").
2. Clicking on a city displays the forecast for that city below the list.
3. The forecast component fetches mock weather data for the selected city using **useEffect** when the city changes.
4. The application includes a button to scroll to the detailed weather section using **useRef**.

This activity will help students combine and practice `useEffect` and `useRef`.

## Tasks

### Step 1: Set Up the Application

**File Structure**:

- `App.jsx`: Main component that manages the selected city and displays the CityList and CityForecast components.
- `CityList.jsx`: Component to list cities and handle user selection.
- `CityForecast.jsx`: Component to display the forecast for the selected city.

### Step 2: Create Mock Weather Data

Define mock data to simulate weather details:

```
const weatherData = {
  NewYork: {
    summary: "Sunny, 25°C",
    details: "Clear skies throughout the day with mild
```

```
temperatures.",
  },
  London: {
    summary: "Cloudy, 18°C",
    details: "Overcast with occasional light rain in the afternoon.",
  },
  Tokyo: {
    summary: "Rainy, 22°C",
    details: "Continuous rain expected throughout the day.",
  },
};
```

### Step 3: Create the CityList Component

- The component displays a list of cities and calls a function when a city is clicked.

### Step 4: Create the CityForecast Component

1. The component receives the selected city as a prop.
2. Use `useEffect` to simulate fetching the weather data for the selected city.
3. Use `useRef` to scroll to the detailed weather section when a "View Details" button is clicked.

### Step 5: Bring it all Together in App.jsx

The main component should manage which city is selected and pass data to children.

### Extensions

- **Handle Invalid City Names**:
  Display a fallback message if the selected city does not match any entry in weatherData.

- **Add a Back Button**:
  In `CityForecast`, add a button that clears the selected city so the user can return to the city list view.
- **Style the Application**:
  Use CSS to improve the layout and appearance of the city list and forecast pages.

## Setup

For this activity, you will need to create your own repository to store code:

1. Create a new repository.
2. Name it: practice-reactWeatherApp-[Your First Name]-[Your Last Initial]
3. Make sure the repository is public.
4. Clone the repository to your local machine.
5. Start working on the code.
6. When you are finished, commit your changes and push them to your repository.

## Submission

Submit the url for your GitHub repo in the field below.